Experiment No. 5

Aim :

To explore the architecture, program structure, and foundational language constructs of Solidity, and gain practical hands-on proficiency in developing, compiling, and deploying smart contracts within the Remix Ethereum IDE using the LearnETH Solidity Basics tutorial module and Remix VM environment.

Theory :

Solidity is a statically typed, contract-oriented programming language designed for developing smart contracts on the Ethereum Virtual Machine (EVM). The LearnETH tutorial introduces fundamental concepts such as data types, variables, variable scope, functions, visibility specifiers, access-control modifiers, constructors, conditional statements, loops, arrays, mappings, structs, and enums. It also explains the use of storage and memory for managing data within smart contracts.

The tutorial further covers function mutability (view and pure), Ether denominations such as Wei, gas fees, and blockchain transactions. These concepts help developers understand how smart contracts read and modify blockchain state, execute operations, and transfer digital assets. Remix IDE provides a browser-based environment to write, compile, debug, test, and deploy Solidity smart contracts using the simulated Remix Virtual Machine (VM).

Steps :

1. Launch the Remix Ethereum IDE and activate the LearnETH tutorial plugin.
2. Navigate to the Solidity Basics course and review the chronological sequence of assignments.
3. Implement and examine primitive data types, variable declarations, and default initializations.
4. Build and test functions with diverse parameter inputs and return outputs, including view and pure mutability.
5. Study access visibility specifiers (public, private, internal, external), custom modifiers, and constructors.
6. Construct and manipulate composite data structures including dynamic arrays, key-value mappings, structs, and enums.
7. Investigate EVM data locations and observe the operational differences between storage and memory pointers.
8. Analyze transaction mechanics, Wei/Gwei/Ether denominations, execution gas costs, and Ether transfer methods.
9. Compile each tutorial contract with the integrated Solidity compiler and verify assignment test assertions.
10. Deploy and interact with the deployed smart contracts using the local Remix VM environment.

Output :

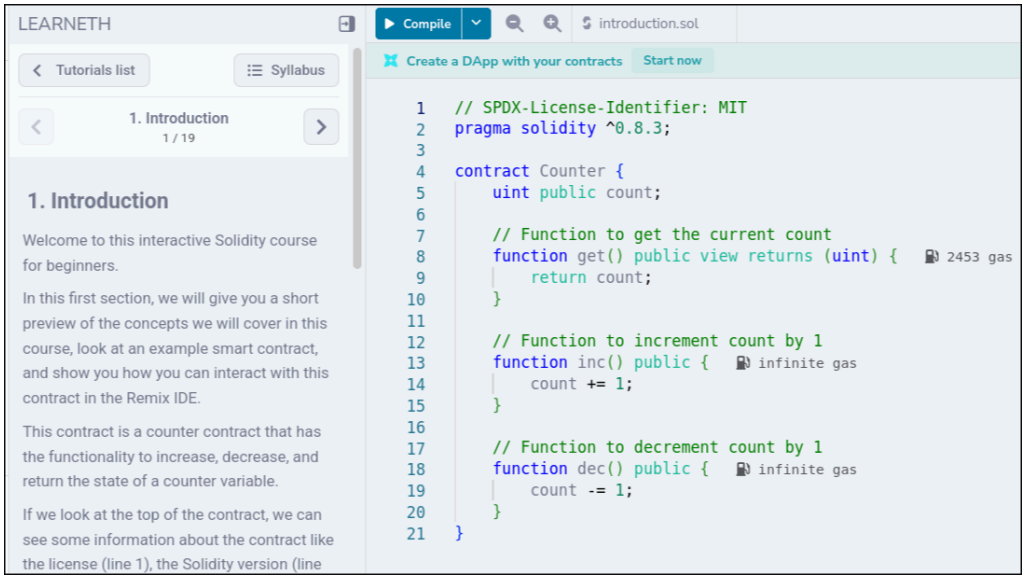


Fig. 1. LearnETH introduction with a basic Counter smart contract

Experiment No. 5

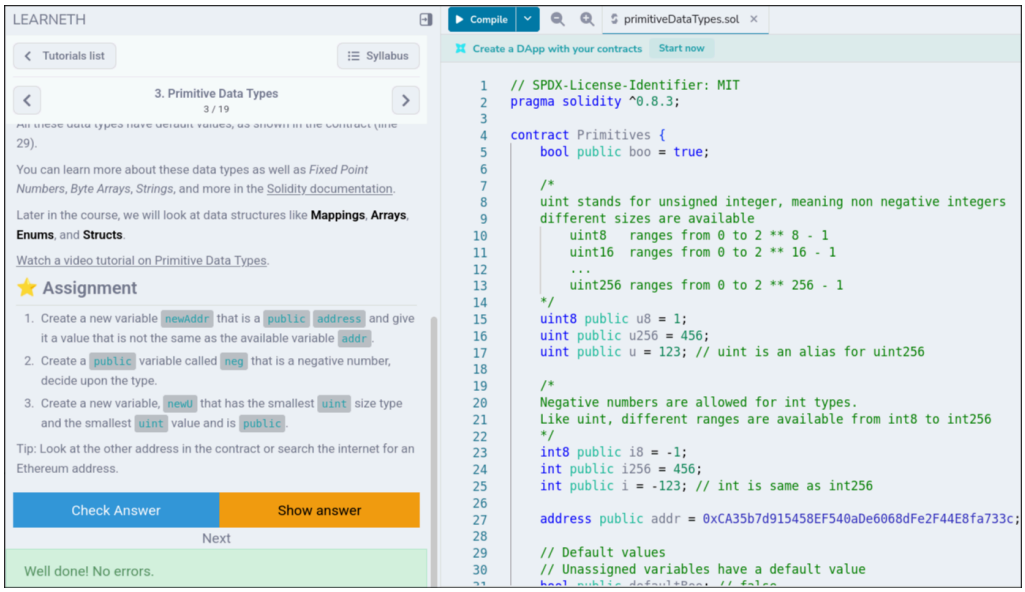


Fig. 2. Basic syntax and contract name state variable assignment

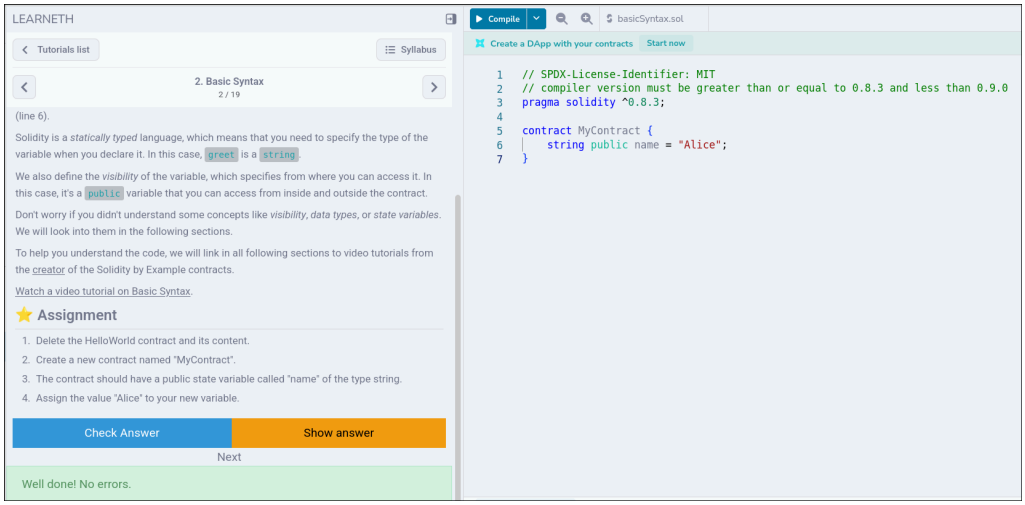


Fig. 3. Primitive data types, integer ranges, and default values assignment

Experiment No. 5

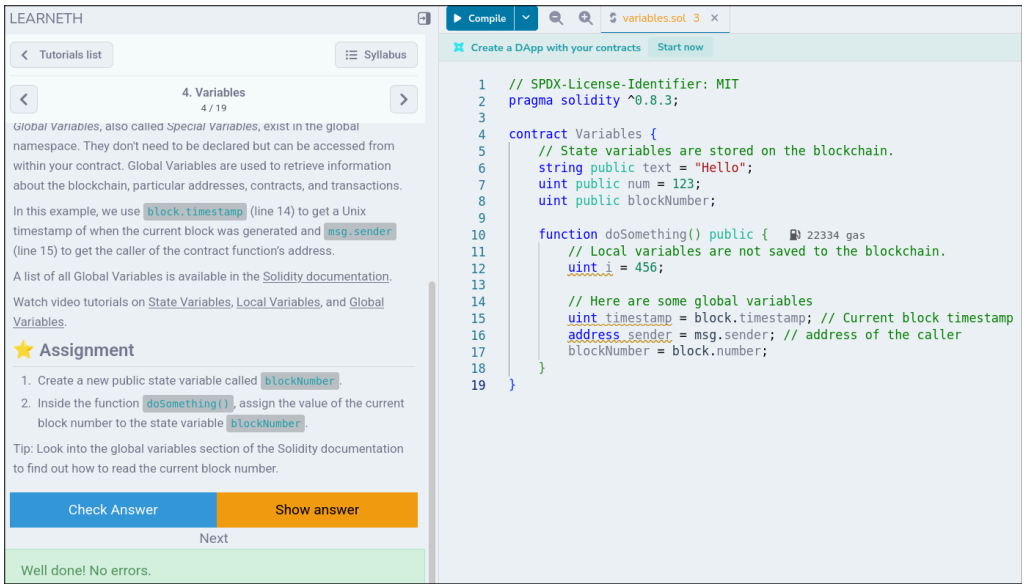


Fig. 4. State, local, and global variables with block number assignment

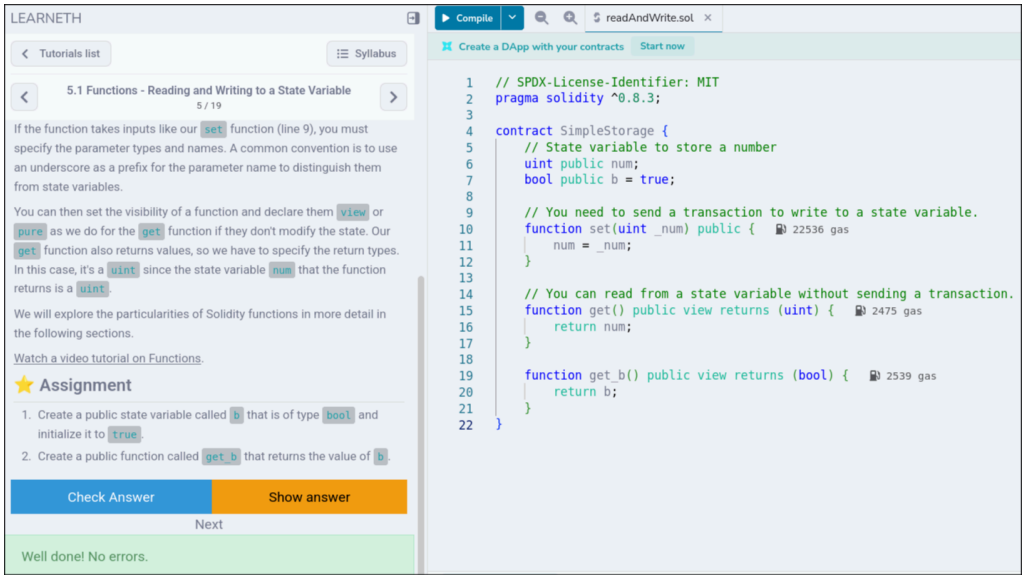


Fig. 5. Reading and writing to state variables in SimpleStorage contract

Experiment No. 5

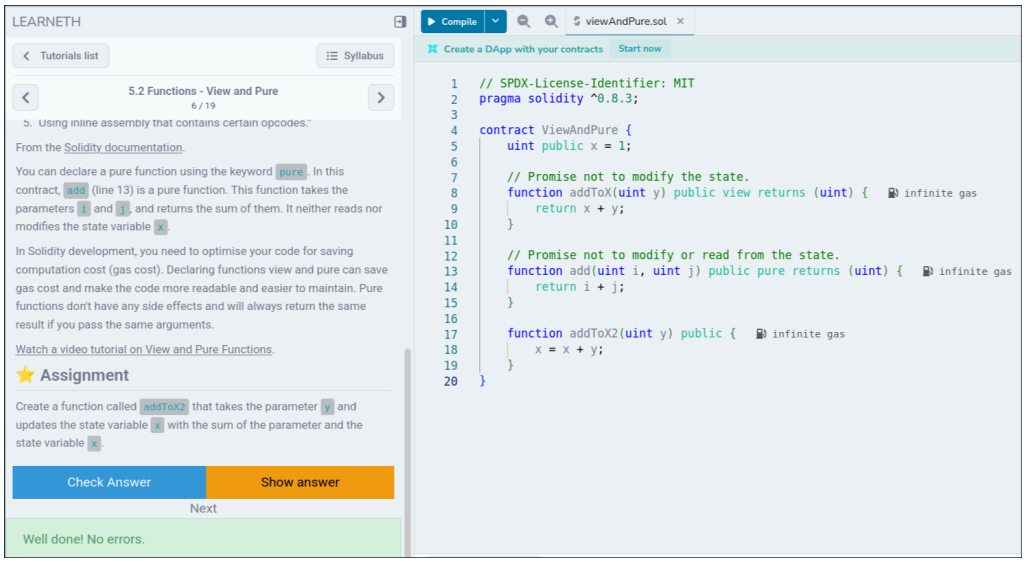


Fig. 6. Pure and view functions with arithmetic addition implementation

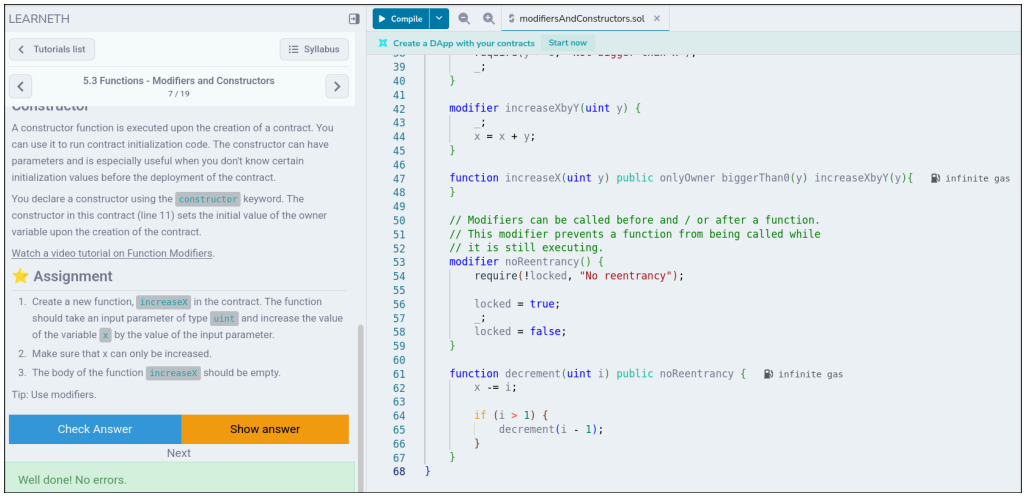


Fig. 7. Function modifiers, constructor initialization, and access control

Experiment No. 5

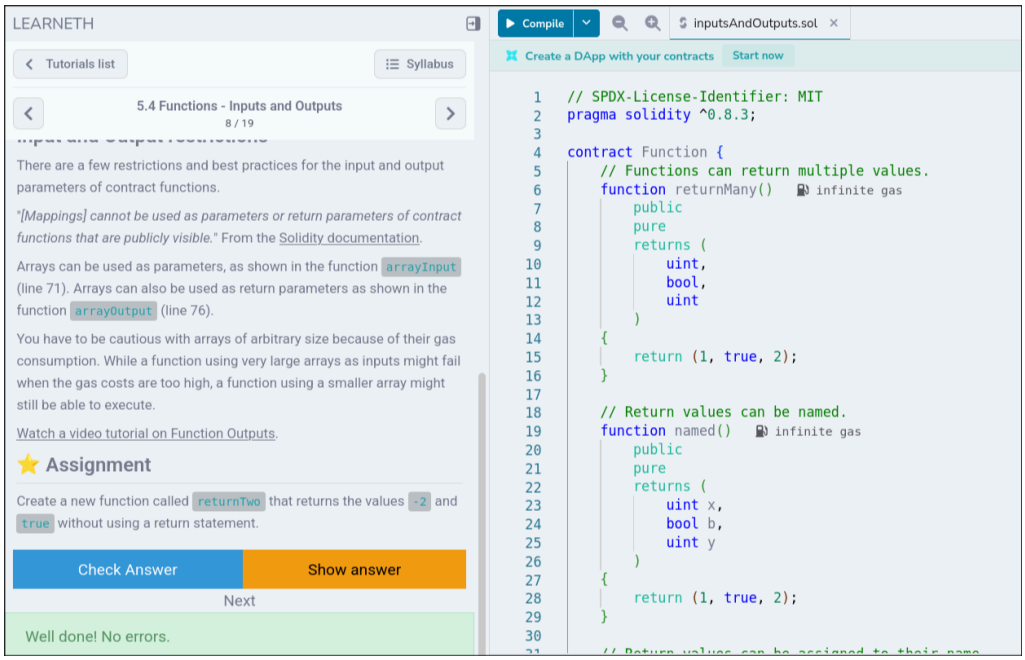


Fig. 8. Function inputs, named return outputs, and destructuring assignment

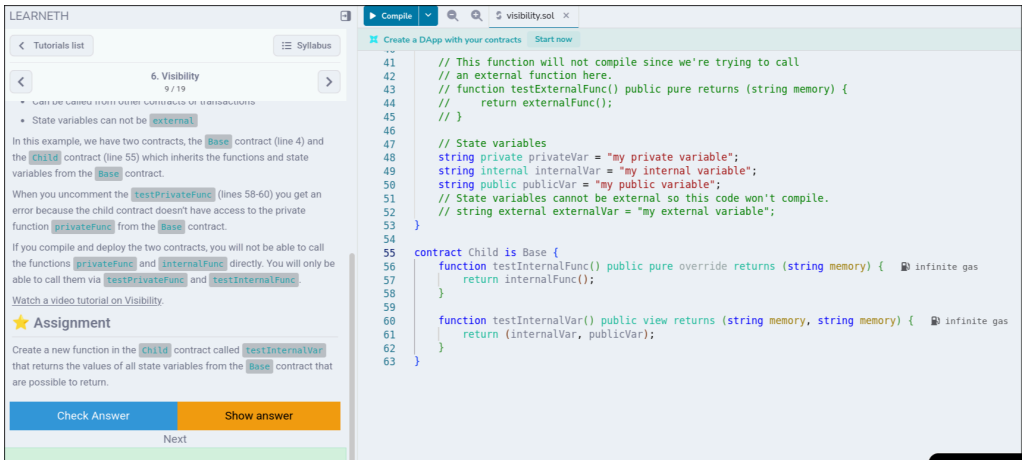


Fig. 9. Visibility specifiers and contract inheritance hierarchy verification

Experiment No. 5

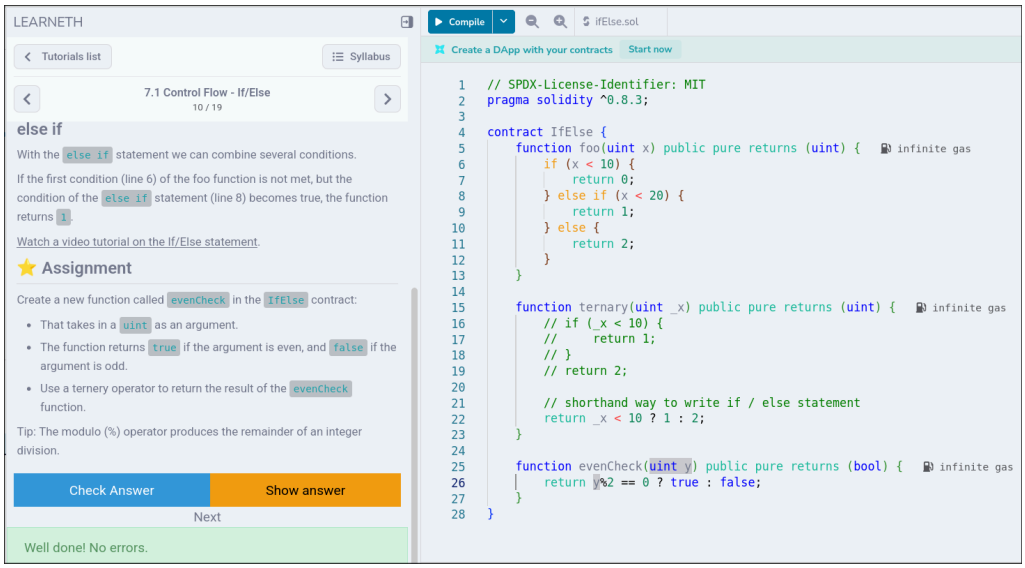


Fig. 10. Control flow with if-else branching and ternary operator assignment

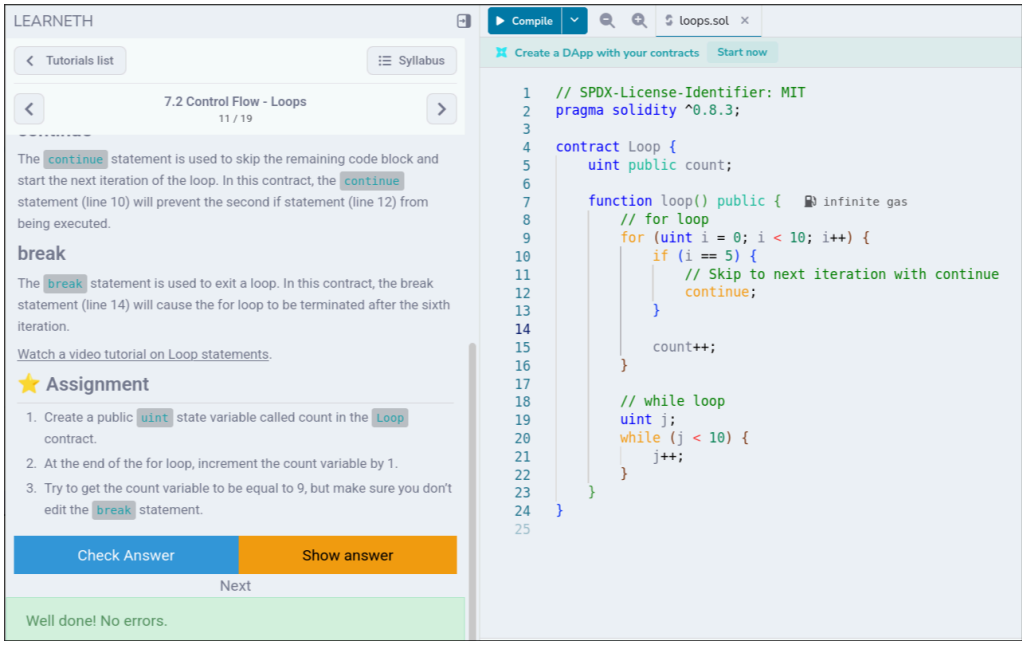


Fig. 11. Iterative loop control flow using continue and break statements

Experiment No. 5

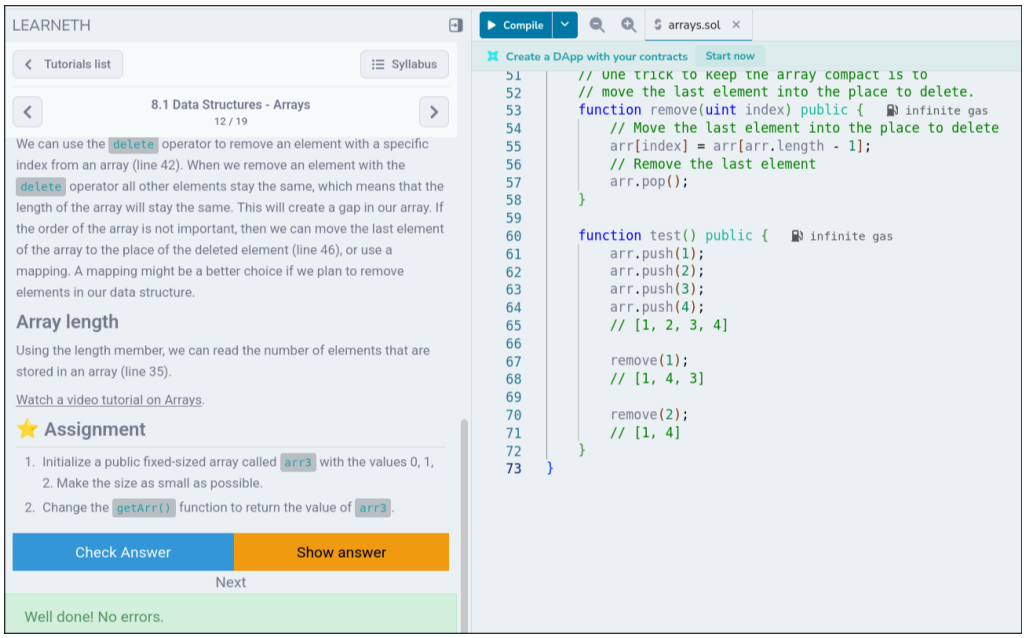


Fig. 12. Fixed and dynamic array initialization and element retrieval

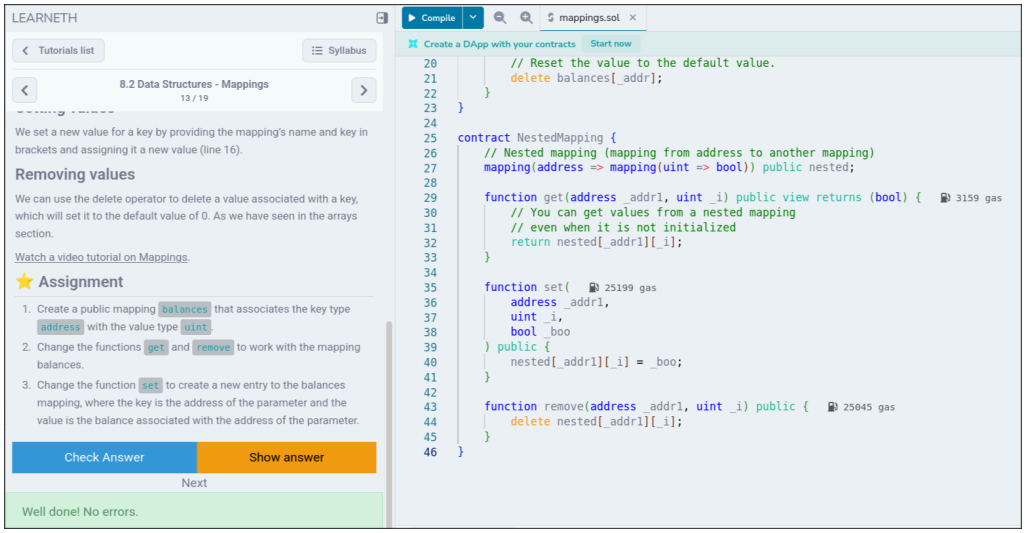


Fig. 13. Hash mappings, balance lookups, and key deletion assignment

Experiment No. 5

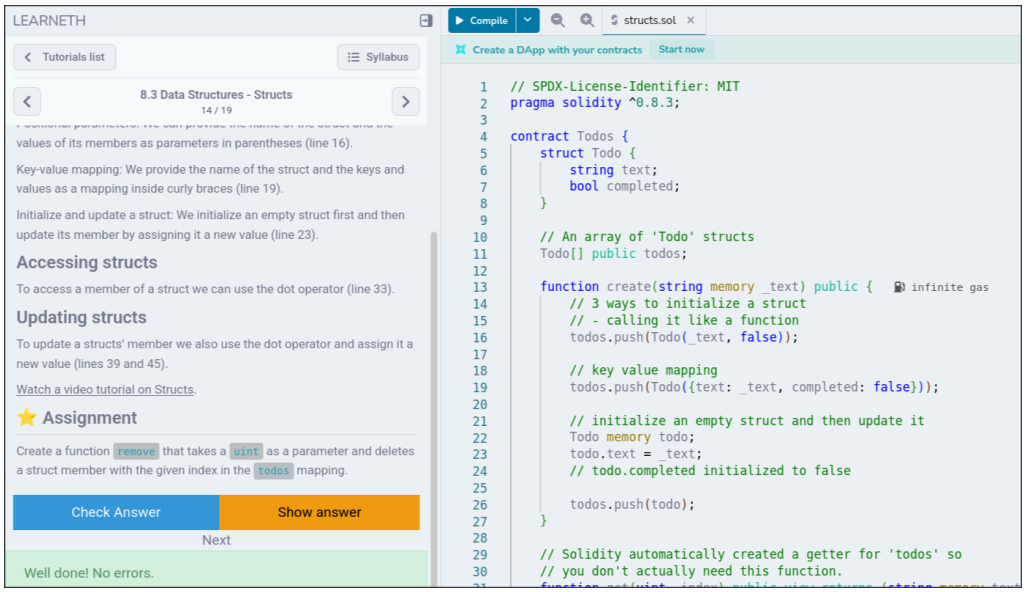


Fig. 14. Struct declarations and task management CRUD operations in Todos

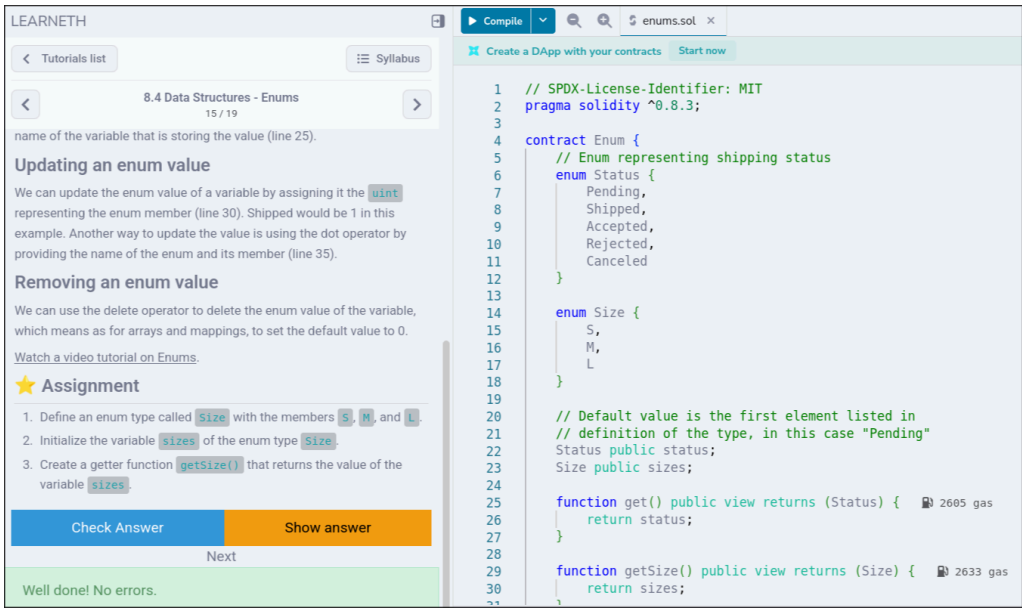


Fig. 15. Enumerations and discrete status state tracking assignment

Experiment No. 5

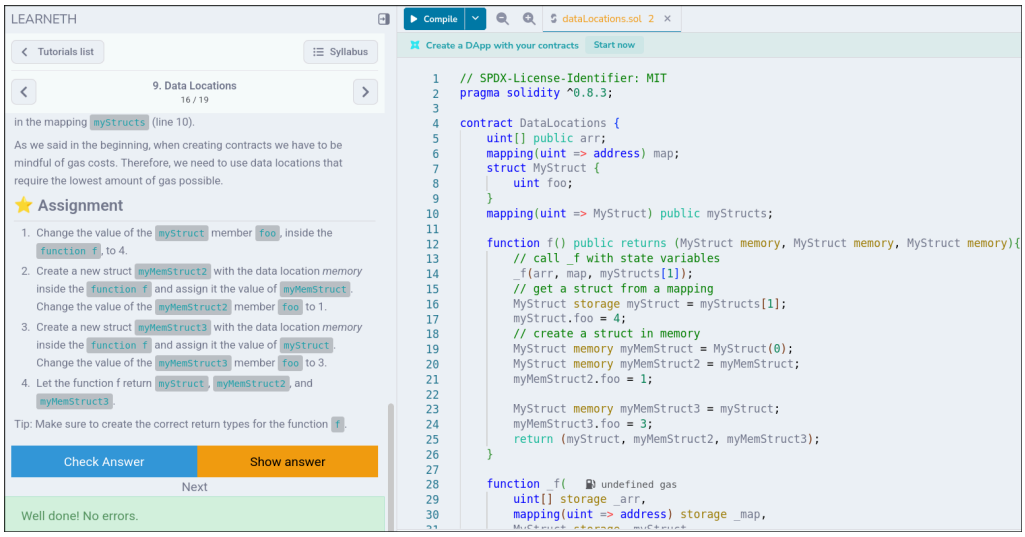


Fig. 16. EVM data locations: storage pointers versus memory copy mechanics

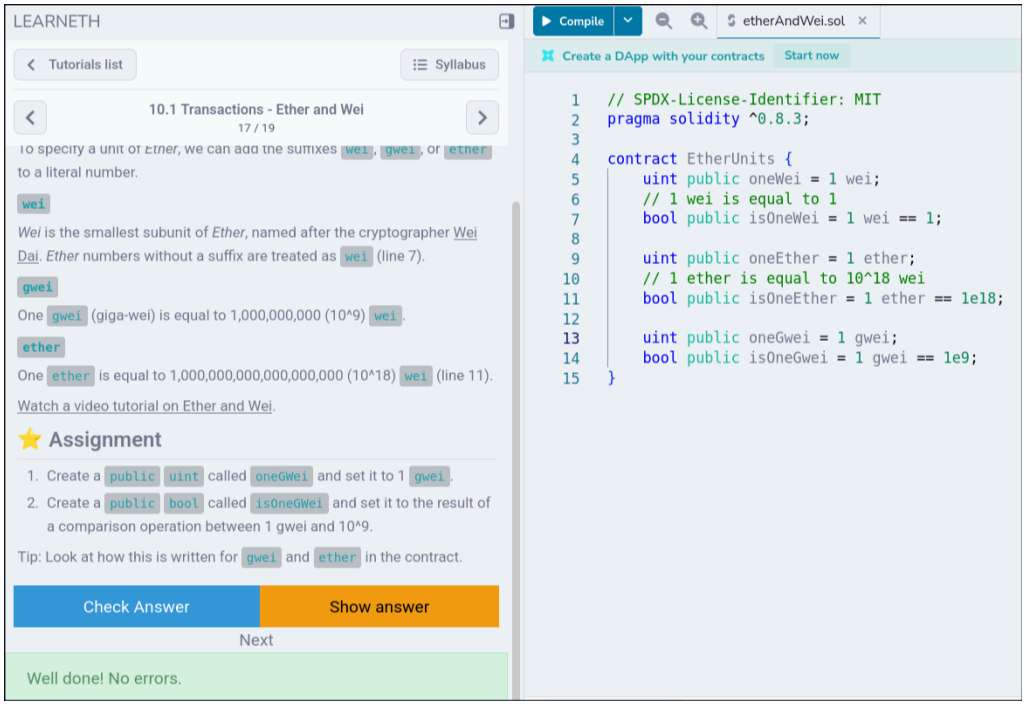


Fig. 17. Ether denominations and Wei conversion comparison assignment

Experiment No. 5

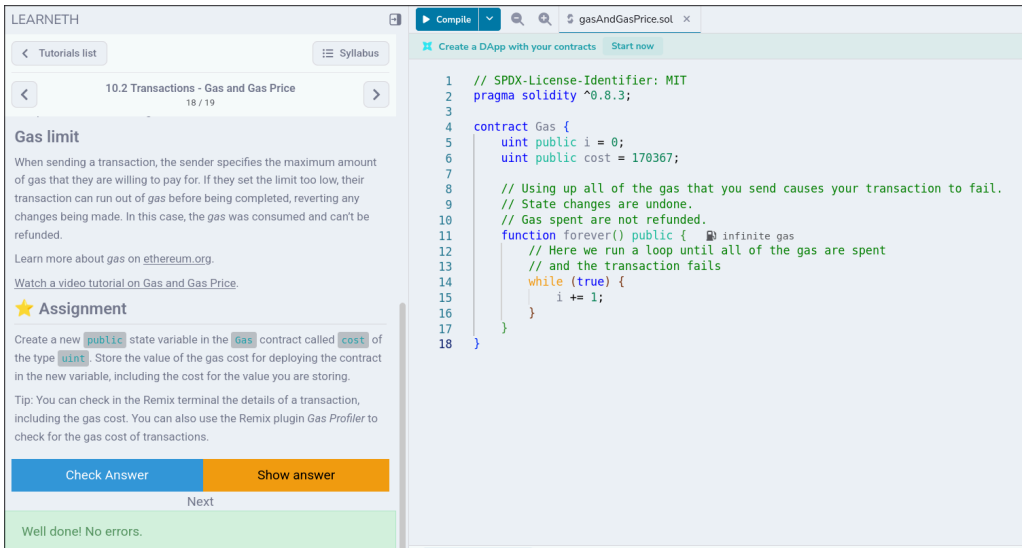


Fig. 18. Gas consumption dynamics and out-of-gas transaction exception

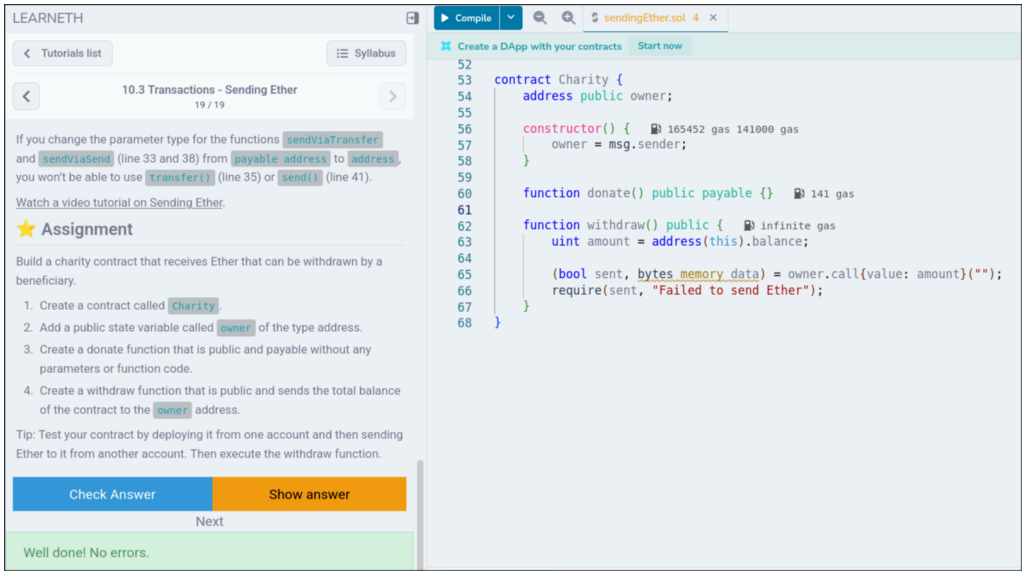


Fig. 19. Sending Ether methods and Charity contract withdrawal implementation

Conclusion

The Solidity Basics tutorial course was systematically completed and verified using Remix IDE. The practical exercises provided a solid conceptual and practical foundation in smart contract program structure, primitive data types, variable scoping, pure and view function mutability, visibility rules, custom access-control modifiers, initialization constructors, branching logic, loop control flow, composite data structures (arrays, mappings, structs, enums), EVM data locations (storage versus memory), and transaction mechanics. The captured screenshots demonstrate the successful compilation, execution, and validation of all 19 assignment test cases. This laboratory experiment delivered comprehensive hands-on exposure to developing, testing, and deploying decentralized smart contracts on the Remix Virtual Machine.